

COMS BC3997: New Directions in Computing (Applied Cryptography)

Lecture 1: Introduction

BARNARD COLLEGE OF COLUMBIA UNIVERSITY

Sept 3, 2025

Welcome to COMS BC3997

New Directions in Computing
(Applied Cryptography)

Who am I?

Eysa Lee (eylee@barnard.edu)

- 2nd year Assistant Professor in the Computer Science Department at Barnard
- PhD from Northeastern, followed by 2 years of postdoc at Brown
 - Couple internships at industry research labs along the way
- My area of research is cryptography
 - Most of my work has been on threshold signatures and privacy-preserving digital credentials



Who are you?

Let's take a moment to introduce ourselves!

What is Cryptography?

- Oxford Dictionary: “the art of writing or solving codes”
 - Greek: *κρυπτός γράφειν* (*kryptós graphein*)
 - English: *hidden writing*

What is Cryptography?

- Oxford Dictionary: “the art of writing or solving codes”
 - Greek: *κρυπτός γράφειν* (*kryptós graphein*)
 - English: *hidden writing*
- For many centuries, focused exclusively on secure communication
 - Consumers were military and intelligence organizations
- Constructions relied on **creativity** and **personal skill**



Historical Cryptography

- People used their intuition to come up with very clever codes that seem to be secure
 - Shift cipher (Caesar's cipher): shift each letter by a predetermined amount
 - Substitution cipher: sample a permutation and apply the permutation to each letter
- ... until someone more clever broke them
 - Shift cipher: Only 26 possible solutions. Try each one!
 - Substitution cipher: Statistical patterns in the English language narrows down likely permutations



Historical Cryptography

- Cryptography and ciphers have a fascinating history
 - Interesting and creative ideas (which are almost all broken by now)
 - Influenced world history (e.g., cryptanalysis of the German Enigma in WWII)
- Much of cryptography up until the 20th century was a **heuristic process**
- What does it mean for something to be “secure”?
 - Is this notion of “secure” useful and realistic?
 - Can we convince ourselves and others that a scheme meets this notion?



By Alessandro Nassiri - Museo della Scienza e della Tecnologia
"Leonardo da Vinci", CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=47910919>

Modern Cryptography

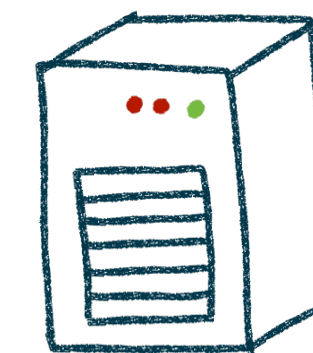
- Late 20th century we started viewing cryptography as a **science**
 - Formalized notions of security
 - Rigorous threat models, firm foundations, and proofs!

What are some examples of how cryptography is used in practice?

What guarantees do we want in these scenarios?

Modern Cryptography

- Late 20th century we started viewing cryptography as a **science**
 - Formalized notions of security
 - Rigorous threat models, firm foundations, and proofs!
- Much much more than secure communication
 - Authentication, anonymous communication + authentication, verifying computations efficiently, secure elections, consensus, computing on secret data without revealing it (*Homomorphic Encryption*), private computation with people you don't trust (*Multiparty Computation*), currency without a central authority (*Blockchain/E-Cash*), and more!
- Used everywhere by everyone (including you!)



Course Philosophy

- This course is meant to act as a bridge between **theoretical crypto** and **crypto used in practice**
- I want you to get **hands-on experience** working with cryptography and to see **how we can use these tools to build cool things!**
- Unlike COMS BC3262 (Intro to Crypto), this course is *much less* about math and proofs
- There's still some math, but we'll focus more on how crypto can be used to do interesting things

What We Will (or at least will try to) Cover

- Security and privacy guarantees of basic cryptographic primitives and how they fit together in larger systems
- Design decisions in practical cryptosystems and the resulting security guarantees of the final system
- Hands-on experience implementing cryptosystems, starting from basic foundational algorithms to advanced constructions

Prerequisites

- **Required:** COMS W3157 (Advanced Programming) or equivalent
 - You are expected to be familiar with basic socket programming, use of command line, and how to use a Makefile
 - Programming assignments must be done in either C++ or Rust as these are two of the most widely used languages for crypto libraries
- **Recommended:** Prior background in cryptography or security is encouraged but not required. We will cover cryptography/security topics throughout the semester

What We Will *Not* (Explicitly) Cover

- Nitty gritty proof details
 - You are encouraged to take Introduction to Cryptography if you want to understand *why* things are secure :)
- Hardware
- How to program
 - You are expected to have developed these skills in your previous classes
- How to write production-ready secure code
 - There's much more to writing secure code than just knowing the algorithms!
 - Our focus will be on basic theory concepts that are important for practitioners, but we will briefly touch on what makes writing secure code so hard.

Course Logistics

Communication

- Course website: <https://www.eysalee.com/courses/f26/bc3997.html>
 - Contains the following links and contact
- EdStem:
 - I will post announcements such as updates to office hours and assignments here
 - You are encouraged to ask questions about assignments, but do not post solutions!
When in doubt, please post privately
- Courseworks: <https://courseworks2.columbia.edu/courses/245292>
- Gradescope: <https://www.gradescope.com/courses/1377863>



Grading Structure

- 15% Participation
- ~~20% In-class Quizzes~~
- 50% Programming Assignments
- 35% - Final Project

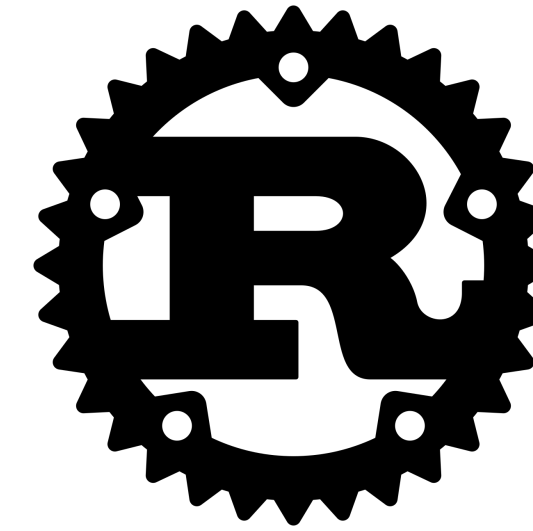


No more quizzes

There are no written exams in the class :)

Programming Assignments (40%)

- ~4-5 programming assignments to be completed in Rust or C++
 - You are **allowed the assistance of AI** for help **debugging** and **syntax** (unless otherwise specified), but **you should not have AI do the entire assignment for you!**
- You will also submit a write-up of design decisions
 - **Random sets of students** will be chosen for **code walks** for each assignment, which will be based on your design document
- Assignments will be downloaded on Courseworks and submitted on Gradescope
 - Assignment 0 will be released soon



Homework Late Policy

- You may submit programming assignments up to 3 days late
 - 10% penalty per day
 - After 3 days the deadline is an automatic 0 (Just submit anything you have!)
- Example:
 - 4 hours past the deadline is a 10% deduction
 - 2.5 days past the deadline is a 30% deduction
 - 37 hours after the deadline is a 0

Final Project (30%)

- Open-ended research-y project!
- Each student will do a deep dive on an advanced topic of their choosing (e.g., Online Voting, Secure Messaging, Anonymous Credentials, Private Information Retrieval, Private Set Intersection, ...)
- Deliverables will be a written report, (optionally) code, and a final presentation in the final week(s) of class
- More information later in the semester

In-class Quizzes (20%)

- We'll have random quizzes
 - These will not be announced ahead of time
 - Except in extenuating circumstances, these cannot be made up
- I'm planning for these to be ways for you and I to gauge how well you are following the material
 - This should not be a source of stress, you are not expected to study extensively for the quizzes

Participation (10%)

- By default everyone gets full credit on participation
- However, if it feels like attendance is dwindling or people aren't engaging, then the grading structure for participation will change

Any questions so far?

We're going to switch to the board
:)